PLATFORM · INTEGRATION · FULL-STACK

임기호

복잡한 운영을 하나의 흐름으로 바꿉니다

레거시, 반복 업무, 데이터 단절을 통합 플랫폼과 자동화로 해결해 온 풀스택 엔지니어입니다.

경력 소개

lasid84@gmail.com

GitHub

limgiho.github.io/portfolio/

15종 이상	90+	약 29% 감소	1,000만+
레거시 시스템 통합	통합 업무 화면	일일 외부 호출 · 항공수출 업무 화면 기준	세금계산서 데이터 이관

01 · CAREER JOURNEY

주요 경력

제조·물류 시스템을 개발하고 운영해 온 경험을 시간순으로 정리했습니다.

2010–2015 — ILJIN Global

MES·ERP 업무 시스템 개발

생산 데이터 수집과 MES·ERP 업무 시스템을 개발하며 제조 현장의 흐름을 익혔습니다.

2016–2022 — ILJIN Global

국내외 공장 MES 고도화

노후 클라이언트를 현대화하고 국내외 공장의 추적성과 실시간 ERP 연계를 구축했습니다.

2022–2023 — (주)케이더블유이코리아

물류 업무 자동화와 데이터 전환

반복 입력, 외부 연동, 여러 DB의 데이터 이관을 검증 가능한 자동화 흐름으로 바꿨습니다.

2023–현재 — (주)케이더블유이코리아

통합 업무 플랫폼 개발 및 운영

15종 이상의 운영 시스템을 분석하고 웹·API·Worker·데이터 경계를 갖춘 단일 플랫폼으로 전환했습니다. 통합 완료 후 운영을 이어가며 해운수입 업무를 새로 전산화하고 있습니다.

통합 업무 플랫폼 개발

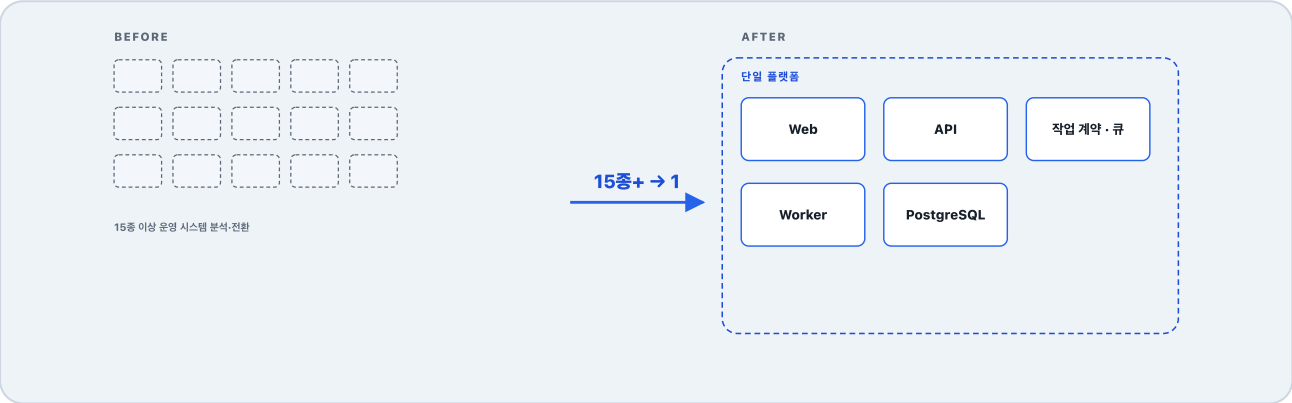
2023.09–2026.08

분산되어 있던 운영 시스템을 웹·API·비동기 실행·외부 연동·데이터·배포 경계로 나누어 하나의 플랫폼으로 통합했습니다. 현재는 통합 플랫폼 운영과 해운수입 업무의 신규 전산화를 진행하고 있습니다.

구현 범위

15종 이상	90+	5개	8종	4단계	변경 앱만
운영 시스템 분석·전환	업무 화면 구현	업무 도메인 분리	외부 시스템 연동	API→Queue→Worker →API 내부 실행	CI/CD 빌드·배포

전환 전후

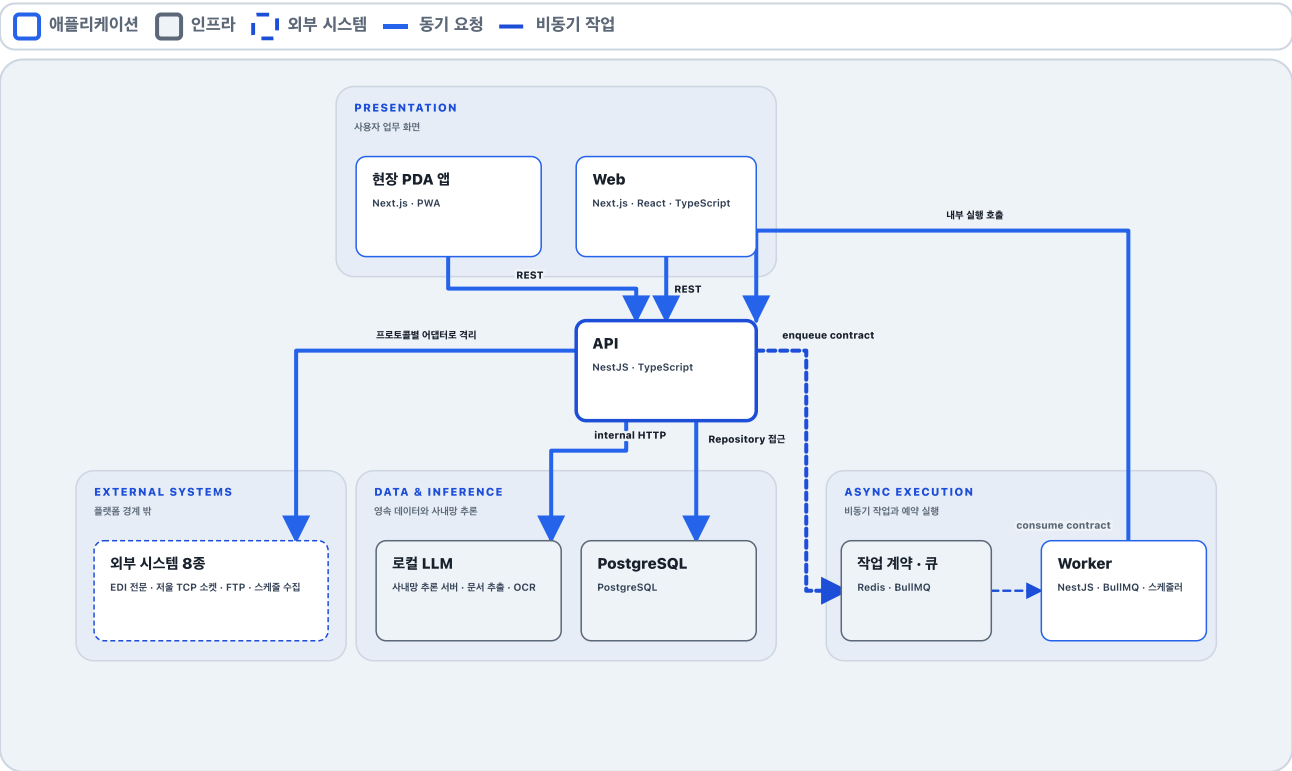


Oracle·MySQL·ODBC로 흩어져 있던 운영 데이터는 DB 구조를 새로 설계해 PostgreSQL 하나로 통합했습니다.

시스템 구성

- 도메인 로직은 API에 모았습니다.
- Worker는 업무 로직을 갖지 않고 API의 같은 유스케이스를 호출합니다.
- 오래 걸리거나 실패할 수 있는 작업은 큐와 Worker로 뺐습니다.
- 프로토콜이 제각각인 외부 연동은 어댑터 안에 가뒀습니다.

구성요소를 누르면 각 판단의 구현과 계약을 볼 수 있습니다.

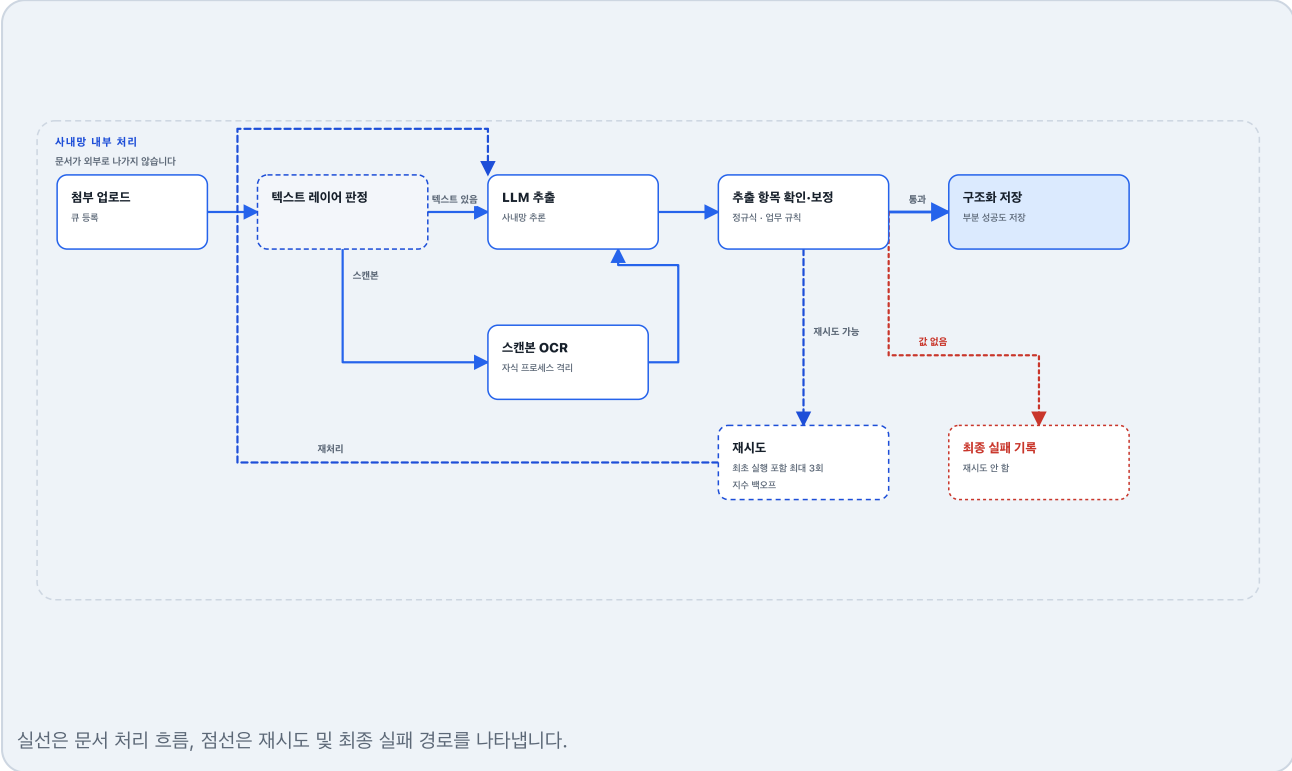


문서 처리

2026년 6~9월, 월 100~600건의 통관 업무에서 담당자마다 달랐던 검토 기준을 정리했습니다. 서류 9종의 55개 대조 항목을 20개 업무 규칙으로 표준화하고 자동 검증했습니다.

DOC·DOCX·엑셀·PDF·스캔본의 파일 형식별 추출과 LLM 호출을 분리하고, 문서별 추출 항목 설명과 few-shot 예시를 설정으로 관리했습니다. 검증 실패 사례와 처리 오류를 기록·분석해 추출 설정과 검증 로직을 개선했습니다.

현업 확인 기준 통관 서류 검토는 건당 8분에서 3분으로 줄었습니다. 인보이스·발주서·FDA 관련 서류에서 추출한 정보로 선적서를 작성하고, 프리알럿 메일 작성·관련 서류 첨부를 연결해 후속 업무에서 하루 약 1시간을 추가 절감했습니다.

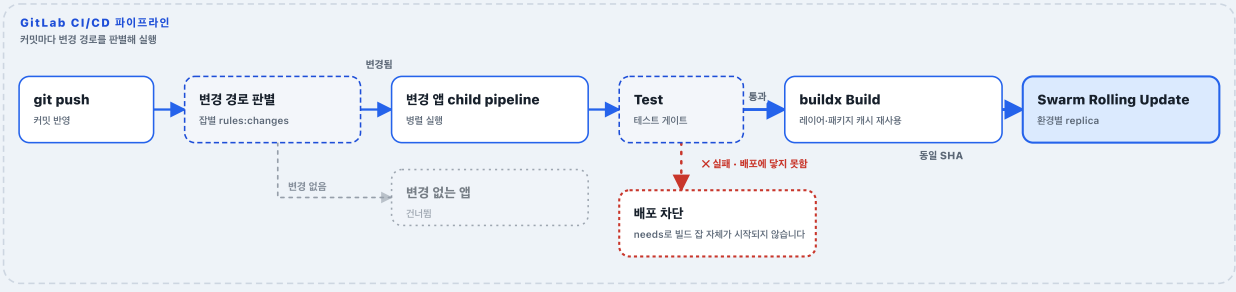


실선은 문서 처리 흐름, 점선은 재시도 및 최종 실패 경로를 나타냅니다.

배포

2025.11~12에 GitLab CI/CD·Docker Swarm 기반 배포 체계를 구축했습니다. 2026.09에는 Kaniko에서 buildx로 전환하고 캐시·러너를 개선해 테스트·빌드·배포 시간을 약 12분에서 6분으로 줄였습니다.

테스트를 통과하지 못한 변경은 그림의 구조상 배포에 물리적으로 닿지 않습니다. 브랜치에 따라 development·staging·production으로 갈리고, replica는 development 1개, staging·production 각 2개입니다.



실패 경로는 배포 차단 노드에서 끊어지고 어떤 배포 노드로도 이어지지 않습니다. 흐린 노드는 변경이 없어 건너뛴 앱입니다.

03 · AUTOMATION

본사 시스템 연동 방식 전환과 처리 결과 검증

2025.06–2025.10

앞 섹션 플랫폼이 연동하는 외부 시스템 8종 가운데 하나입니다. 화면이 쓰던 호출 규격을 알아내 업무 단위로 다시 묶었습니다.

문제

반복 입력을 Puppeteer로 자동화해 월 약 1,200시간의 업무 감소 효과를 추산했습니다. 다만 화면 자원 로딩이 100Mbps 사내 회선의 60~70Mbps를 점유해 연동 방식의 개선이 필요했습니다.

제약

외부에 공개된 연동 API가 없어 브라우저 요청을 분석해야 했습니다. 요청이 성공해도 화면에서 처리한 업무 결과와 같다는 보장이 없어 별도의 검증이 필요했습니다.

접근

화면 없이 업무 데이터만 주고받는 HTTP 연동 구조를 설계하고 동료 1명과 구현했습니다. 400개 이상 필드와 입력 조건을 공통 로직으로 정리하고, 코드 처리 결과와 실제 화면 입력 결과를 저장·비교했습니다. 전환 후 회선 사용량은 대체로 5Mbps 미만입니다.

75개 외부 호출

화면이 보내던 요청을 하나씩 그대로 재현합니다.

12종 업무 워크플로

여러 HTTP 요청을 하나의 업무 흐름으로 묶어 재사용합니다.

항공 운송장 갱신 한 건

조회 여섯 번, 재계산 두 번, 저장 한 번의 HTTP 요청을 순서대로 실행해 하나의 업무 흐름으로 묶었습니다.

기존 업무 자동화의 절감 효과 추산

기존 자동화 효과 산정 근거 ↗

HTTP 전환 이전에 구축한 반복 입력 자동화의 효과입니다. 당시 부서별 처리량과 작업 시간을 기준으로 월 약 1,200시간의 업무 감소 효과를 추산했습니다.

부서	기능	월 절감 예상(시간)	월 처리량
항공수출	운임 코드·원가 자동 업로드	**250**	3,000건
항공수출	운임 청구금액 이상 자동 검출	**100**	3,000건
항공수출	항공 운송장 확정 처리	**125**	1,500건
항공수출	출하 자동 마감	**80**	500건
항공수출	전자문서 자동 전송	**40**	500건
항공수입	해외 정산 운임 자동 계산·업로드	**165**	1,000건
항공수입	국내 정산 운임 자동 업로드	**230**	1,400건
항공수입	출발지 부대비용 확인	**70**	2,000건
해운수입	운임 코드·원가 업로드	**80**	1,000건
경리	운임 코드·원가 업로드	**80**	1,000건
전산	환율 자동 입력	**7**	22회
합계 · 11종 · 5개 부서		**1,227**	

호출 기록 분석과 반복 조회 개선 · 2026.10

항공수출 업무 화면의 일일 외부 호출 약 7천 → 5천 건 · 약 29% 감소

개선 대상 선정

전체 업무 화면의 외부 호출을 기록·분석해 개선 대상을 찾았습니다. 항공수출 화면에서는 문서·템플릿 검색이 해당 화면 호출의 약 94%였습니다. 연속 행 이동 중 중간 대상까지 파일 목록을 조회하고, 상세 화면을 다시 열 때마다 변경이 드문 템플릿을 재조회하고 있었습니다.

첨부파일 관리 패널 개선

여러 화면에서 사용하는 첨부파일 관리 패널에 0.5초 디바운스를 적용해 연속 행 이동 시 마지막 대상만 조회하도록 했습니다. 최초 열기와 명시적 새로고침은 즉시 처리하고, 이전 대상의 늦은 응답은 제외했습니다.

변경이 드문 데이터 캐시

템플릿 검색 결과를 Redis에 1시간 캐시했습니다. 빈 결과도 재사용하되 오류는 저장하지 않고, Redis 장애 시 본사 시스템을 직접 조회하도록 했습니다.

04 · WRITING

기록

이틀 동안 운영 API가 다섯 번 죽었다

힙·RSS와 로그·배포 버전·조회 조건을 대조해 대량 조회 경로를 찾았습니다. 500건 단위 조회로 개선하고, 응답 처리 모사 실험으로 메모리 추이를 확인했습니다.

2026.08 장애 분석·개선 · 수만 건 일괄 조회 경로 개선 · 응답 처리 모사 · 힙·RSS 검증

velog에서 읽기

Next.js + BFF 인증 흐름 총정리

액세스 토큰을 메모리에, 리프레시를 HttpOnly 쿠키에 두는 구성이다. 새로고침 복원과 401이 겹칠 때의 재발급까지 정리했다.

액세스는 메모리 · 리프레시는 HttpOnly 쿠키 · 401에 single-flight 재발급

velog에서 읽기

CI/CD 적용기 2편

rsync와 PM2로 시작한 배포를 컨테이너와 Swarm으로 옮긴 과정이다. docker socket 없이 이미지를 굽고, 롤링 업데이트와 SHA 롤백까지 붙였다.

docker socket 없이 Kaniko 빌드 · start-first 롤링 업데이트 · SHA 태그로 롤백

velog에서 읽기

05 · ADDITIONAL WORK

직접 만들고 운영한 제품



댓글 필터 · Android & Web

브라우저와 단말에서 댓글 분류·본문 요약을 수행해 서버 비용과 데이터 외부 전송을 줄인 1인 개발 프로젝트입니다.

React Native · Next.js · TypeScript · Transformers.js · WebLLM · Web Worker

Android · Web · 개발기



액션독

게임 코어 엔진을 계층별로 설계하고 인증, 글로벌 랭킹, 광고까지 연결해 출시한 3라인 매치 게임입니다.

React Native · Expo · TypeScript · Supabase · AdMob

Google Play



HealthDog

운동 기록·조회·통계와 데이터 동기화, 구독 결제를 구현해 운영 중인 개인 프로젝트입니다.

Google OAuth · Firebase · Supabase · RevenueCat

Google Play

시스템 구성 상세

API의 도메인 경계와 프론트엔드·로컬 LLM의 책임 및 입출력을 정리했습니다.

구현 상세 · 01

API

NestJS · TypeScript

5개 도메인의 동기 요청 처리, Worker가 위임하는 배치 실행까지 포함

구현

도메인 서비스가 데이터·외부 시스템 구현체를 직접 알지 않도록 포트와 어댑터로 분리했습니다. 외부 연동이 요청·수집·수신 세 방향이고 프로토콜이 제각각이라 하나로 억지 추상화하지 않고 각 구현을 어댑터로 격리했습니다. Worker가 호출하는 내부 배치 실행 엔드포인트도 같은 도메인 서비스를 재사용해, 실행 로직을 별도 앱으로 중복시키지 않았습니다.

구조

```
<모듈>/
├─ adapter/
│  ├─ controller/          HTTP 진입
│  └─ dto/
├─ application/
│  └─ usecase/
│     └─ <유스케이스>/
│        ├─ *.usecase.ts   업무 기능 하나
│        ├─ dto.ts
│        └─ mapper.ts
└─ infra/
   └─ repository/          읽기 · 쓰기 분리
```

운영

요청 검증, 권한, 공통 오류 변환과 추적 정보를 한 경계에서 처리했습니다. 외부 오류는 내부 오류 형식으로 바꾸고 재시도 가능 여부를 구분했습니다.

입력

Web·PDA REST 요청 · Worker 내부 배치 실행 요청

출력

Repository 호출 · 로컬 LLM 호출 · 외부 연동 · Queue job · 배치 실행 결과

계약

DTO validation · Symbol port · 공통 job package · 단기 인증 토큰

배포

API 컨테이너 수평 확장

설계 배경

초기 API는 Express로 자유도 높게 만들었고, 경계가 나뉘어 있지 않아 한 곳을 고치면 다른 곳이 깨졌습니다. NestJS와 클린 아키텍처를 도입해 모듈마다 방향을 고정했습니다.

더 읽기

이틀 동안 운영 API가 다섯 번 죽었다

구현 상세 · 03

Web

Next.js · React · TypeScript

5개 업무 도메인의 90여 개 화면

구현

화면 단위 로컬 슬라이스와 공통 도메인 슬라이스를 분리하고, 서버 상태와 화면 상태의 책임을 나눴습니다. 기능이 많은 화면은 그 안에서 기능별로 다시 나눴습니다.

구조

```
<도메인 화면>/
├ _container/        화면 조합 · 상태 연결
├ _features/         업무 기능 묶음
│  └ <기능>/
│     ├ container/  기능 상태 · 이벤트
│     ├ view/       기능 표현
│     └ hooks/      기능별 훅
├ _hooks/            화면 공통 훅
└ _store/            화면 상태
```

운영

화면과 기능이 폴더 단위로 갈려 있어 수정 범위가 그 폴더 안에 머뭅니다.

입력

사용자 입력 · 조회 조건 · 첨부 문서

출력

REST 요청 · 업무 결과 화면 · 파일 다운로드

계약

TypeScript DTO · HTTP status · 공통 오류 형식

배포

Web 컨테이너 독립 배포

설계 배경

프론트도 경계가 나뉘어 있지 않아 한 곳을 고치면 다른 화면이 깨졌습니다. API와 같은 방식으로 클린 아키텍처를 옮겨보려 했지만 상태 관리와 훅이 레이어 구분에 얹히지 않았습니다. 여러 구성을 시험한 끝에 기능 슬라이스로 나누는 FSD로 정리했습니다.

구현 상세 · 05

로컬 LLM

사내망 추론 서버 · 문서 추출 · OCR

문서 추출을 사내망 안에서 처리

구현

추론 서버를 사내망 장비에 띄우고 HTTP로 호출합니다. 고객사 문서라 망 밖으로 나갈 수 없어 외부 API를 쓰지 않았습니다. JSON Schema로 LLM 출력 구조를 지정하고, 일부 추출 항목을 정규식과 업무 규칙으로 확인·보정합니다. 서류 간 누락과 불일치는 별도의 업무 규칙으로 검사합니다.

구조

텍스트 레이어 판정 · 스캔본 OCR(전체 페이지 → 표 영역 재인식) · JSON 응답 파싱 · 반복 추출 테스트

운영

PDF에 텍스트 레이어가 있으면 그대로 읽고, 스캔본만 OCR로 처리합니다. 렌더 해상도는 실물 문서로 반복 측정해 정했습니다. 300dpi에서 나던 오인식이 400dpi에서 사라졌고 처리 시간은 거의 같았습니다.

입력

수출입 문서 · 추출 스키마

출력

검증된 구조화 데이터 · 실패 사유

계약

Internal HTTP · JSON Schema

배포

사내망 추론 서버 독립 운영